

Lezione 17: Java Streams API e Programmazione Funzionale

Questo notebook raccoglie tutto il codice della Lezione 17 (`MavenDate`):

- `src/main/java/it/oop/ui/MainStream.java` (Elaborazione con Java Streams)
- `src/main/java/it/oop/ui/MainDate.java`
- Tutte le classi del modello `Date` , `ItalianDate` , `AmericanDate` , `TimeStamp` , `DateInterval` , eccezioni

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/  
├── codice-commentato/  
│   └── Lezione17/  
│       ├── MavenDate  
│       │   ├── pom.xml  
│       │   └── src  
│       │       ├── main  
│       │       │   ├── java  
│       │       │   │   ├── it  
│       │       │   │   │   ├── oop  
│       │       │   │   │   │   ├── core  
│       │       │   │   │   │   │   ├── AmericanDate.java  
│       │       │   │   │   │   │   ├── BirthDay.java  
│       │       │   │   │   │   │   ├── Date.java  
│       │       │   │   │   │   │   ├── DateInterval.java  
│       │       │   │   │   │   │   ├── DatePair.java  
│       │       │   │   │   │   │   ├── FormattedDate.java  
│       │       │   │   │   │   │   ├── FormattedDateConverter.java  
│       │       │   │   │   │   │   ├── ItalianDate.java  
│       │       │   │   │   │   │   ├── OrderedPair.java  
│       │       │   │   │   │   │   ├── Pair.java  
│       │       │   │   │   │   │   ├── Time.java  
│       │       │   │   │   │   │   └── TimeStamp.java  
│       │       │   │   │   │   ├── exception  
│       │       │   │   │   │   │   ├── IllegalDateException.java  
│       │       │   │   │   │   │   └── OrderdPairException.java  
│       │       │   │   │   │   └── ui  
│       │       │   │   │   │       ├── Date.java  
│       │       │   │   │   │       ├── MainDate.java  
│       │       │   │   │   │       └── MainStream.java  
│       │       │   └── resources  
│       │       └── test  
│       │           ├── java  
│       │           │   ├── it  
│       │           │   │   ├── oop  
│       │           │   │   │   ├── core  
│       │           │   │   │   │   └── TestItalianDate.java
```

Argomenti trattati:

- Concetto di `Stream<T>` : sequenze di elementi con computazione lazy
- Sorgenti di Stream: `Stream.iterate(...)` , `Stream.generate(...)` , `Stream.of(...)`
- Operazioni intermedie: `filter(...)` , `map(...)` , `limit(...)`
- Operazioni terminali: `toList()` , `forEach(...)` , `count()`
- Reference a metodi (`System.out::println`)
- Calcolo di numeri primi e trasformazione funzionale di collezioni di date

1. Classi del Modello `Date` ed Eccezioni

```
In [1]: class IllegalDateException extends RuntimeException {  
        public IllegalDateException(String message) { super(message); }  
    }
```

```

class Date implements Comparable<Date> {
    protected int day, month, year;
    public Date(int day, int month, int year) {
        this.day = day; this.month = month; this.year = year;
    }
    public int getDay() { return day; }
    public int getMonth() { return month; }
    public int getYear() { return year; }
    @Override public String toString() {
        return String.format("y%dm%dd%d", year, month, day);
    }
    @Override public int compareTo(Date other) {
        int d = this.year - other.year;
        if (d != 0) return d;
        d = this.month - other.month;
        if (d != 0) return d;
        return this.day - other.day;
    }
}

abstract class FormattedDate extends Date {
    protected final String format;
    public FormattedDate(int day, int month, int year, String format) {
        super(day, month, year); this.format = format;
    }
    public abstract String prettyPrint();
}

class ItalianDate extends FormattedDate {
    public ItalianDate(int day, int month, int year) { super(day, month, year, "dd/mm/yyyy"); }
    @Override public String prettyPrint() { return day + "/" + month + "/" + year; }
    @Override public String toString() { return day + "/" + month + "/" + year; }
}

class AmericanDate extends FormattedDate {
    public AmericanDate(int day, int month, int year) { super(day, month, year, "mm/dd/yyyy"); }
    @Override public String prettyPrint() { return month + "/" + day + "/" + year; }
    @Override public String toString() { return month + "/" + day + "/" + year; }
}

```

2. Classe `MainStream` ed Esecuzione

```

In [2]: import java.util.List;
import java.util.stream.Stream;

class MainStream {
    public static void main(String[] args) {
        // Generazione di stringhe di lunghezza dispari crescenti: "a", "aaa", "aaaaa", ...
        Stream<String> stream = Stream.iterate("a", s -> s + "a")
            .filter(s -> s.length() % 2 == 1)
            .limit(10);
        List<String> list = stream.toList();
        System.out.println(list.toString()); // [a, aaa, aaaaa, aaaaaaa, aaaaaaaaa, aaaaaaaaaa, aaaaaaaaaa, aaaaaaaaaa, aaaaaaaaaa, aaaaaaaaaa]

        // Generazione di 10 numeri primi casuali compresi tra 0 e 19
        List<Integer> list2 = Stream.generate(() -> (int) (Math.random()*20))
            .filter(i -> isPrime(i))
            .limit(10)
            .toList();
        System.out.println(list2.toString()); // [1, 1, 2, 11, 7, 13, 3, 17, 5, 5]

        // Pipeline funzionale su oggetti Date: filtraggio, mapping e stampa
        Stream.of(new ItalianDate(15, 1, 2025), new Date(16, 1, 2025), new ItalianDate(2, 2, 2025))
            .filter(d -> d instanceof ItalianDate)
            .map(d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear()))
            .forEach(System.out::println);
        // Stampa in output riga per riga:
        // 1/15/2025
        // 2/2/2025
    }
}

```

```
private static boolean isPrime(int n) {
    return n == 0 ? false : Stream.iterate(1, i -> i + 1)
        .limit(n)
        .map(i -> n % i)
        .filter(i -> i == 0)
        .count() <= 2;
}

}

MainStream.main(null);
```

[a, aaa, aaaaa, aaaaaaa, aaaaaaaaa, aaaaaaaaaa, aaaaaaaaaaaa, aaaaaaaaaaaaa, aaaaaaaaaaaaaa, aaaaaaaaaaaaaaa]

[2, 13, 11, 1, 2, 11, 3, 2, 11, 11]

1/15/2025

2/2/2025